Machine Learning Course Project-I

Flight Price Prediction (Regression)

Team Name: Carrot

Name	Roll number	Contribution
Aradhya Chakrabarti	MT2026012	• Preprocessing for the flight competition (data loading, cleaning and feature engineering) • Models Tested: linear regression (OLS), polynomial regression, Ridge, Lasso, decision trees, random forests (with cross-validation, evaluation and hyperparameter tuning)
Ankit Lamba	MT2026XXX	`<contribution>`
Kunal Yadav	MT2026XXX	`<contribution>`
Kshitij Beri	MT2026XXX	`<contribution>`

Source code

The full code for this competition is in the project repository: `https://github.com/TimeATronics/AIT511_Project1`

1 Problem and dataset

We are given a regression problem: predict the ticket price (in USD) of a US domestic flight. The training file `train.csv` has 18,076 rows and 12 columns, and the test file has 4,520 rows and 11 columns. The columns are airline, flight, source_city, destination_city, departure_time, arrival_time, stops, flight_class, duration (hours), days_left, and the target price. The evaluation metric is R^2.

We have used the following models: linear regression (ordinary least squares), polynomial regression, Ridge (L2), Lasso (L1), decision trees, and random forests.

2 Data loading

We begin by loading the three CSV files (training, test and the sample submission) with pandas, printing their shapes, the submission header and the first rows, and building a small overview frame with the row count, column count, duplicate rows and missing cells. Both files turn out to be complete: there are no duplicate rows and no missing cells in either file, so no imputation is needed in practice. We still keep a median/mode imputer inside the pipeline so that the code would not break if missing values appeared later.

3 Exploratory data analysis

3.1 Data distribution check (skewness)

For EDA, the first thing we check is the shape of the target, since it decides which error metric is sensible and whether a transformation could help the linear models. We compute the descriptive statistics along with skewness and kurtosis, and plot the raw prices next to $\log(1 + \text{price})$ (Figure 1). From the histogram we see that most tickets are concentrated below USD 400, while a handful of fares reach USD 8,996 against a median of only USD 234. The computed skewness (7.07) and kurtosis (170.25) quantify how strongly these few expensive tickets pull the distribution away from the typical fare. Since the models are scored with squared error, the same tickets can dominate the score, and the optimal regression function under squared error is the conditional mean $f^*(x) = \mathbb{E}[Y \mid X = x]$, which a skewed target makes sensitive to a few rows.

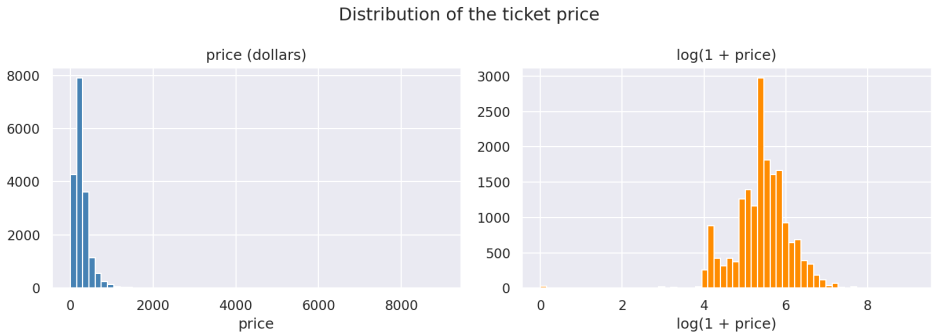


Figure 1: Price distribution in raw dollars and after $\log(1 + \text{price})$.

3.2 Feature-wise analysis

Next we look at the two numeric features and at how price behaves across the categorical ones. The code prints the descriptive statistics of `duration` and `days_left`, their correlations with price, and draws boxplots for the numeric columns and for price across seating class, stops and airline (Figure 2). Accordingly, we observe the following:

- Business tickets are clearly higher in price, with a median near USD 320 against USD 194 for Economy, and Business holds the very expensive fares up to USD 8,996.
- Non-stop flights have a median near USD 214, while one-stop and two-or-more itineraries are around USD 347 and USD 370. Only 193 flights have two or more stops, so that group is relatively small.
- Airline medians range from about USD 168 for Frontier up to about USD 260 for United, Delta and Alaska. Southern Airways Express appears exactly once in the training data, so nothing general can be learned for it.
- `duration` ranges from 0.87 to 28.65 hours, though a few flights run much longer than the rest, and `days_left` covers 1 to 44 days. Neither relates strongly to price: the linear correlations are $+0.281$ and -0.229.

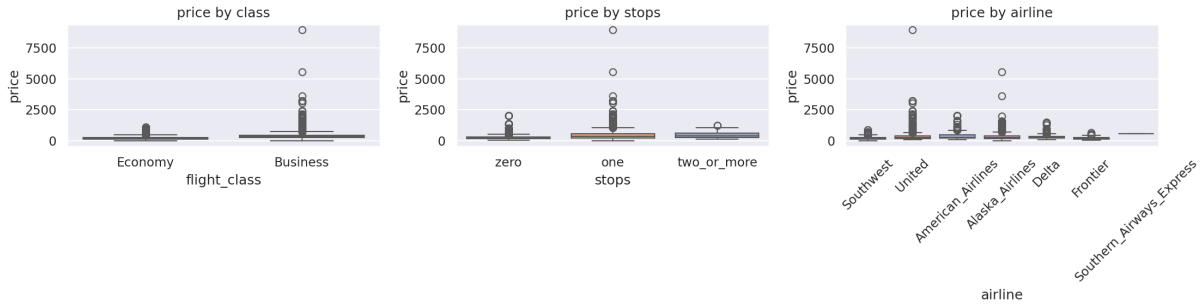


Figure 2: Price by seating class, number of stops and airline.

3.3 Correlation analysis

We then check the correlations between the numeric columns. The heatmap block shows that no pair is strongly linearly related, and the largest correlation with price is +0.28 for `duration`. A linear model trained only on the numeric columns therefore has little to work with, which is why the categorical columns are encoded and why tree-based models do well in Section 8.

3.4 High-cardinality columns

Finally, we inspect the categorical cardinalities. `flight` has 1,123 distinct codes, and 33 of them appear only in the test file. Treating it as a category would create over a thousand sparse columns and the unseen codes would have no coefficients at all, so we replace it with engineered features in Section 5.

4 Data cleaning

For cleaning, we find 25 training rows that carry a price of 0. Since a ticket cannot cost nothing, these look like entry errors and we remove them; 18,051 rows remain for training. The test file is left untouched, and the `id` column is set aside to line up the submissions.

5 Feature engineering

We build three features from existing columns, with one function applied to train and test:

1. `airline_code` and `flight_number` from the flight code. The code "WN 3596" contains the carrier ("WN") and a serial number, and airlines tend to reuse numbers for the same route, so the number can carry route information that the city columns alone do not show.

2. `stops_ordinal`: `stops` is one-hot encoded during preprocessing for the regression models, and mapping the three levels to 0/1/2 adds a compact numeric column that keeps the ordering.

3. `route`: source and destination are merged into one category. The same city pair shares a fare level, and a single merged column can help the tree-based models split on particular city pairs.

6 Data preprocessing

All preprocessing is built with a scikit-learn `ColumnTransformer` and used inside a `Pipeline`, so during cross-validation every transformation is fitted on the training folds only and the validation fold stays untouched.

- Numeric columns are imputed with the column median and standardised for the models that need it. Scaling matters for the penalised models: the Ridge and Lasso penalties act on the size of the coefficients, so features on different scales would otherwise be treated unequally. Trees split on thresholds and use raw values.
- Categorical columns are imputed with the most frequent value and one-hot encoded, with `handle_unknown="ignore"` so an unseen category produces an all-zero row instead of an error at prediction time.
- Columns with more than 100 levels are dropped by the encoder; `flight` is the only such column.

7 Model evaluation

For targets y_i and predictions $\hat{y}_i = f(x_i; w)$, we report

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}, \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|,$$

and

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Minimising the squared error on the training folds is the objective of OLS, Ridge and Lasso (Section 9), while the reported RMSE is computed on the withheld fold, so the two measure different things: fit on the data the model sees, and generalisation on the data it does not. R^2 is a monotone transformation of MSE on the same data, so RMSE and R^2 rank models identically. MAE is a different loss that none of the fitted models minimise, so it acts as an independent check on the ranking.

For validation we split the training data into $K = 5$ folds, hold each fold out once while the model trains on the remaining $K - 1$ folds, and average the five scores: $\frac{1}{K}\sum_{k=1}^{K} e_k$. The finalists are re-checked with 5 folds repeated 3 times (15 scores each). A fixed random state (42) is used for every fold split and every model, so the results repeat.

8 Model family direct comparisons (baseline)

We first train each model family once with reasonable settings. The families are: ordinary least squares (OLS)

$$w^\star = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y};$$

polynomial regression (least squares on squared and interaction terms of the encoded features); Ridge

$$w^\star = (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I})^{-1}\mathbf{X}^\top \mathbf{y};$$

Lasso (L1 penalty, coefficients can be set exactly to zero); a decision tree; and a random forest (trees on bootstrap samples with random feature subsets, averaged). The code blocks build these pipelines, run 5-fold cross-validation for each and draw the comparison chart (Figure 3).

Model	CV RMSE
Random forest (300, sqrt)	128.97
Ridge (L2, $\lambda = 1$)	153.64
Lasso (L1, $\lambda = 0.1$)	154.30
Linear regression (OLS)	154.82
Decision tree (unrestricted)	158.25
Polynomial regression (degree 2)	177.25

The random forest is already better than the rest (around 128 RMSE against about 154 or higher for the other families). Polynomial regression without a penalty is worse than plain linear regression, a first sign that the expansion needs a penalty before it becomes useful. The unrestricted tree overfits, scoring about 158, close to the linear models.

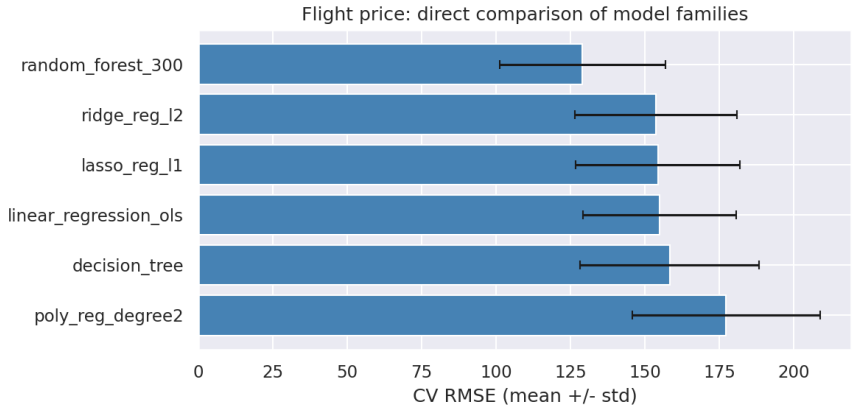


Figure 3: Direct comparison of the model families (CV RMSE).

9 Hyper-parameter tuning

9.1 Polynomial regression: optimal degree

Since the polynomial expansion is the natural way to give a linear model curvature, we tune its degree first. The degree controls how much curvature and interaction the model can represent: degree 1 is a straight line, degree 2 adds squares and pairwise products, and degree 3 adds triple products. We loop over degrees 1 to 3, fit OLS and Ridge ($\lambda = 1$) on each expansion and plot the curves (Figure 4).

Polynomial degree	CV RMSE, L2 ($\lambda = 1$)	CV RMSE, no penalty
1	153.6	154.8
2	136.7	177.3
3	158.0	380.9

Degree 2 without a penalty is worse than degree 1 at 177.3 RMSE, and degree 3 collapses to 380.9 RMSE: with this much flexibility the model fits individual training rows, noise included, and the validation error grows sharply. The same degree-2 expansion with an L2 penalty gives 136.7 RMSE, which is the best raw-price linear model, so we carry the degree-2 expansion forward for the regularised variants.

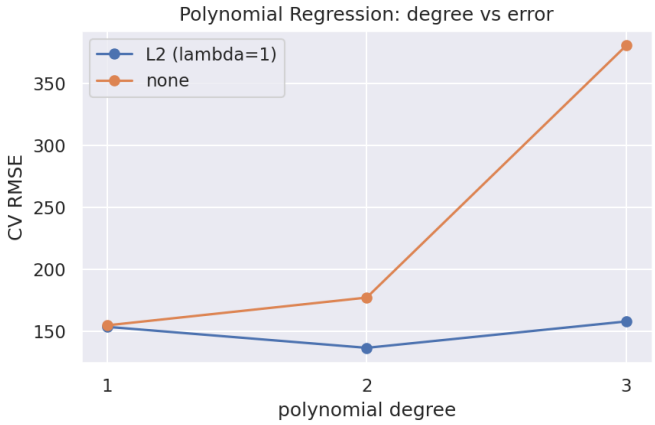


Figure 4: Polynomial degree against CV error, with and without an L2 penalty.

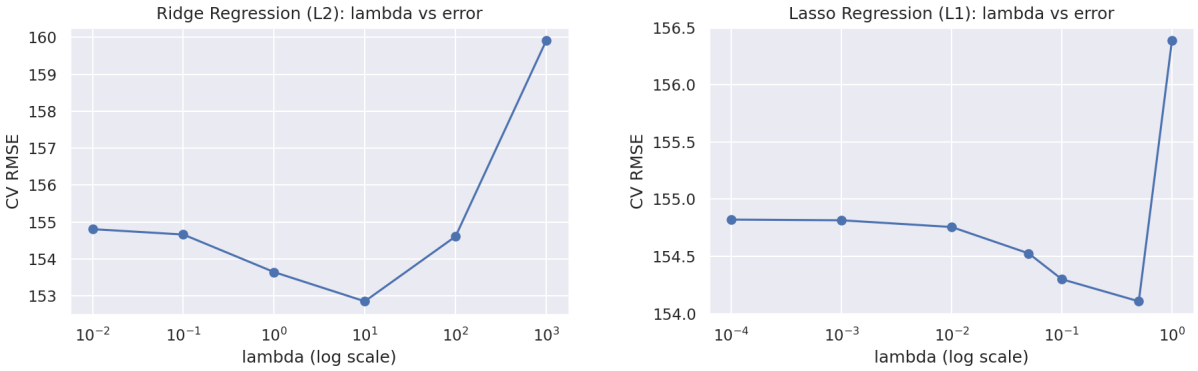
9.2 Ridge and Lasso regularisation

Ridge minimises the squared error plus $\lambda\|w\|_2^2$, which gives the closed form above, and larger λ shrinks the coefficients more. We sweep $\lambda \in \{0.01, \dots, 1000\}$ and plot the curve (Figure 5, left). The curve is flat between $\lambda = 1$ and 100 (152.9 to 154.6 RMSE), with the best single value at $\lambda = 10$ (152.9). Only at $\lambda = 1000$ does the penalty become strong enough to underfit visibly (159.9).

Lasso minimises the squared error plus $\lambda\|w\|_1$. For coefficient w_j the optimality condition is

$$\frac{2}{n}\,\mathbf{x}_j^\top(\mathbf{y} - \mathbf{X}w) = \lambda\, s_j, \qquad s_j \in \partial|w_j|,$$

so a coefficient is set to zero whenever $\left|\frac{2}{n}\mathbf{x}_j^\top(\mathbf{y} - \mathbf{X}w)\right| \le \lambda$. We sweep λ from 10^{-4} to 1, refit the best value and count the surviving features. Lasso tops out at $\lambda = 0.5$ with 154.1 RMSE, slightly behind Ridge, and keeps 42 of the 63 encoded features non-zero, which is the L1 selection behaviour in practice. At the smallest λ values the solver printed convergence warnings, which makes sense because the target is on a dollar scale.



(a) Ridge **(b)** Lasso

Figure 5: Regularisation curves.

9.3 Decision tree: pre-pruning and cost-complexity pruning

An unrestricted tree grows until its leaves are pure, so we tune both routes to control it. For pre-pruning we tune four limits together in a grid block: maximum depth, minimum samples per leaf, minimum samples needed to split a node, and the maximum number of leaves. The grid

peaks at depth 12 with a minimum leaf size of 5, around 128.8 RMSE. Fully grown trees are the worst row at about 158, and shallow depth-4 trees underfit at 149 to 163 depending on the leaf size.

For post-pruning we grow the tree fully and cut it back with the cost-complexity criterion

$$R_\lambda(T) = R(T) + \lambda|T|,$$

where $R(T)$ is the training error and $|T|$ the number of leaves, so λ is the price paid per leaf. Candidate values come from the pruning path of a fully grown tree and are tuned with the same cross-validation. Pruning back to 66 leaves ($\lambda = 66.2$) gives about 148 RMSE, worse than the pre-pruned tree at 128.8, hence we keep the pre-pruned tree (Figure 6).

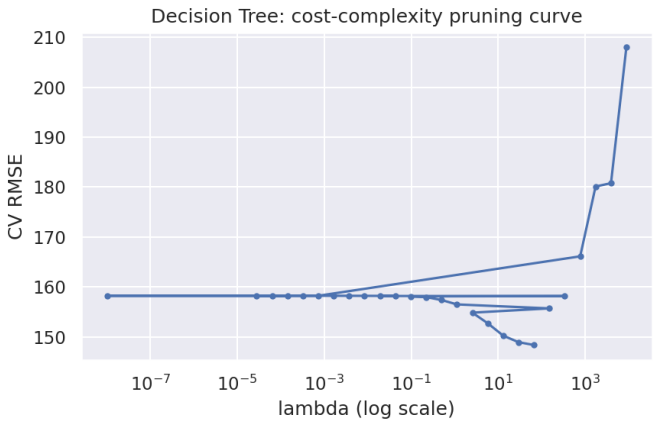


Figure 6: Cost-complexity pruning curve.

9.4 Random forest

The random forest averages many decorrelated trees. We vary the number of trees and the minimum leaf size, keep the features per split at `"sqrt"` (about 10 seconds per fit against 90 seconds when all 36 encoded features are tried), and leave the depth unconstrained because the leaf size already limits the trees. Every configuration with leaf size 3 scores around 121.5 RMSE, while leaf size 1 has nearly 129. Doubling the number of trees from 200 to 400 changes the score by a fraction of an RMSE unit, so we keep the version with 400 trees and leaf size 3.

10 Experiment: regression on log(price)

Since the target is heavily skewed, we test whether training on a transformed target helps the linear models. A handful of tickets cost more than USD 8,000 while the median is USD 234, so with squared error those rows carry an outsized weight. We wrap the polynomial Ridge in a `TransformedTargetRegressor` with `func=log1p` and `inverse_func=expm1`, which fits the model on relative errors and converts the predictions back to dollars before scoring. The sweep tries degree 1 and 2 with $\lambda \in \{0.1, 1, 10, 100\}$ and re-evaluates the winner under R^2.

The best configuration is degree 2 with $\lambda = 10$, at about 125.0 RMSE and $R^2 = 0.6824$, against 136.7 for the degree-2 Ridge on the raw price. The linear models gain from the log target because the expensive tickets no longer dominate the loss, while the random forest does not need it because its leaf averages on raw dollars already match the squared-error metric. Even after this gain the random forest (about 121.4 RMSE) stays ahead, so the log-price Ridge is the strongest linear model rather than the overall winner.

11 Experiment: ensemble of random forest and polynomial ridge

Since we have two very different models available, the Random Forest with axis-aligned splits and the log-price polynomial Ridge with a smooth global fit, we test whether averaging them helps. The code block builds two ensembles with `VotingRegressor`: the random forest with the log-price Ridge, and the same pair plus a plain Ridge as a third member; a voting regressor predicts the average of its members. The two-member average scores about 120.0 RMSE, slightly better than random forest alone at 121.4, while the three-member average scores about 125.7. Since the gap is fairly low, in favour of lesser complexity, we keep the single random forest model as the best.

12 Model diagnostics

12.1 Predicted vs actual (out-of-fold)

We collect out-of-fold predictions for the random forest and the log-price Ridge and plot them against the actual prices (Figure 7). Both panels show the same pattern: the dense cloud of points sits close to the diagonal for ordinary fares (up to roughly USD 1,500), and both models fall below the line for the expensive tickets, meaning they under-predict the rare high fares. The random forest predictions rarely go above USD 2,000 even when the actual price is far higher, which is expected: a tree can only average leaf values, and expensive tickets are too rare to form their own confident leaves. The log-price Ridge stretches a little further into the mid-range but still flattens at the top.

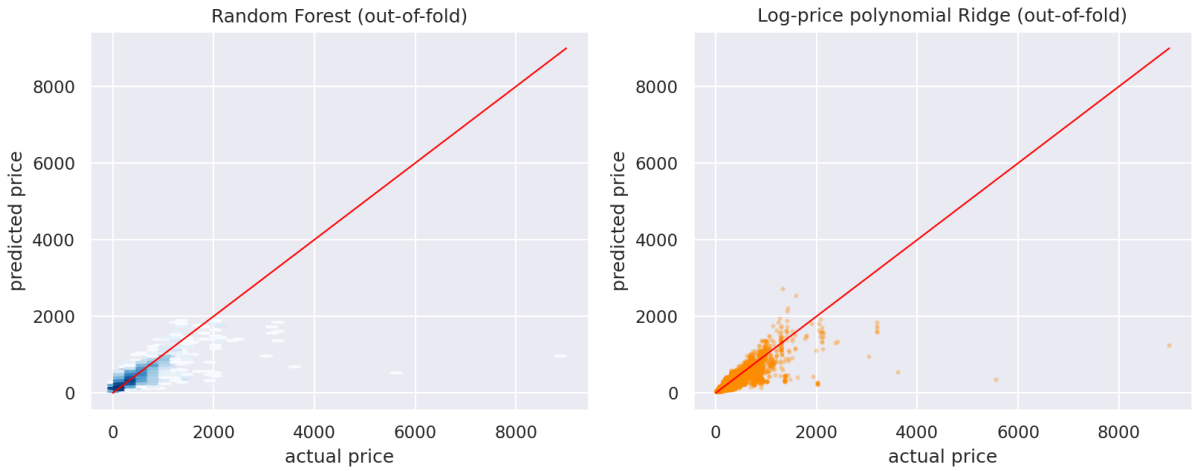


Figure 7: Out-of-fold predicted vs actual price.

12.2 Residual analysis (random forest)

We then compute the residuals $e_i = y_i - \hat{y}_i$ for the random forest and plot the histogram and the residuals against predicted values (Figure 8). The histogram is centred close to zero (the printed mean is -0.18), with a small number of large positive residuals. The scatter plot shows that these large positive residuals occur only where the model predicted a high price. In other words, most predictions are slightly conservative and the largest errors come from the rare expensive tickets that the leaf averages cannot reach.

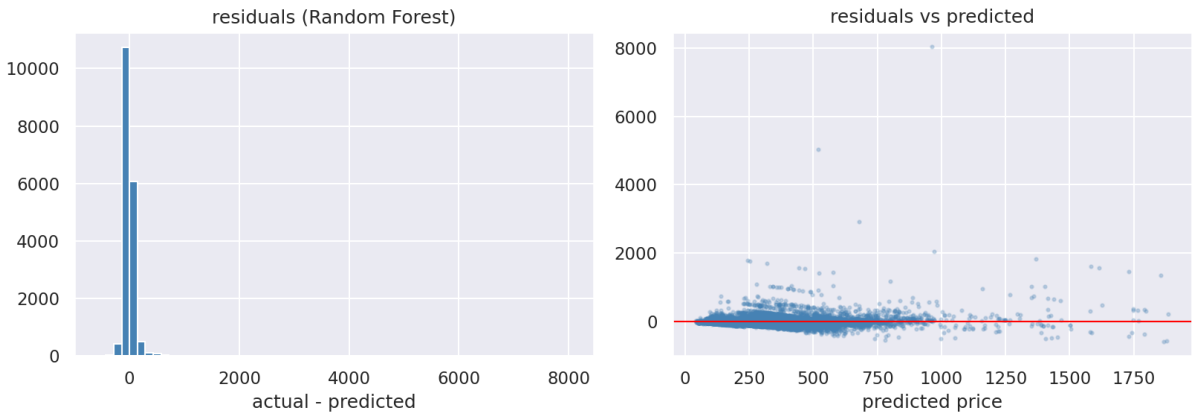


Figure 8: Random forest residuals: histogram and residuals against predictions.

12.3 Feature importance (random forest)

Finally for diagnostics we refit the random forest on all training rows and draw the 15 largest impurity importances (Figure 9). The booking window, the flight length and the seating class are the features which affect the price the most. The engineered columns are genuinely used: `flight_number` (0.05), `stops_ordinal` (0.05), the stop indicators, and the Sacramento-San Jose route (0.036) all appear before the remaining categories.

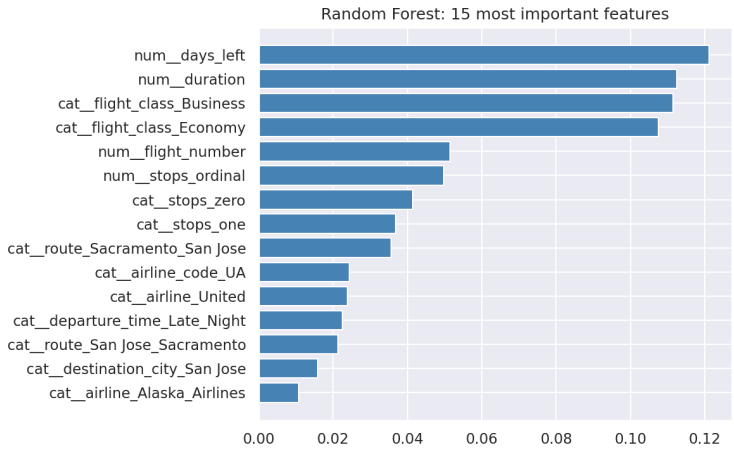


Figure 9: Random forest feature importances.

13 Cross-validation of shortlisted models

Since a single 5-fold estimate can shift with the fold split, we re-evaluate the three finalists with 5 folds repeated 3 times (15 validation scores each): the random forest, the pre-pruned tree and the log-price Ridge. The random forest maintains the best average (121.30 $\pm$ 32.18), the pre-pruned tree stays about 8 RMSE behind it (130.01 $\pm$ 31.75), and the log-price Ridge is between the two (124.86 $\pm$ 30.69). The ordering from the tuning sections survives the different fold splits, so the random forest choice does not depend on one particular partition of the data. The wide variation itself is a property of the skewed target, not of the procedure.

14 Metrics comparison (full table)

We finally evaluate one representative configuration per family under all three metrics in a single cross-validation pass, so the numbers are directly comparable.

Model	RMSE	MAE	R^2
Random forest (400, sqrt, leaf 3)	121.37	50.97	0.7003
Log-price polynomial Ridge (deg 2, $\lambda = 10$)	124.96	55.30	0.6824
Decision tree (depth 12, leaf 5)	128.82	49.31	0.6624
Polynomial Ridge (deg 2, $\lambda = 1$)	136.67	60.86	0.6120
Decision tree, cost-complexity pruned	148.39	65.21	0.5428
Ridge (L2, $\lambda = 10$)	152.85	80.32	0.5244
Lasso (L1, $\lambda = 0.5$)	154.11	80.33	0.5168
Linear regression (OLS)	154.82	80.42	0.5107
Polynomial regression (deg 2, no penalty)	177.25	62.56	0.2742

Table 1: Flight price: full metrics comparison.

From the table we see that the random forest model is best on RMSE and R^2 (about 121.4 RMSE and 0.70 R^2) and second on MAE (51.0), behind only the pre-pruned tree (49.3). The log-price polynomial Ridge is the best linear-family model (about 125.0 RMSE, 0.68 R^2), and the plain polynomial and unregularised variants are the weakest ones. MAE tells a slightly different story from RMSE: the tree tracks typical fares well but pays for its larger errors on the expensive tickets, which RMSE punishes more heavily. The random forest is the only model that is near the top of both.

15 Submission creation and best model

For the submissions, we fit every model on all training rows and write one CSV per model, aligned to the competition's sample file by the id column. The best model is also written as `submission.csv`. The models of the first comparison have all been uploaded to the competition with a description of the model and its cross-validated metric, and the retuned random forest from the next section carries the new best public score.

Table 2: Best model and its submission.

Final model	CV score	Public LB	Submission file
Random forest, 1000 trees, 50% features, leaf 3	R^2 0.7160	0.75904	`submission.csv`

The retuned random forest lifts the best public score from 0.75521 (the original 400-tree model) to 0.75904. The public scores of the first comparison follow the cross-validation ranking: log-price Ridge 0.70155, pre-pruned tree 0.69389, pruned tree 0.64657, polynomial regression 0.63352, Ridge 0.55503, Lasso 0.54922, OLS 0.53131, and the random forest plus Ridge ensemble 0.74533.

16 Testing the random forest with more trees and more features per split

In the earlier sections we settled on a random forest with 400 trees and max_features='sqrt', where each split looks at the square root of the total number of columns. Here we test the same model with more trees and with max_features raised to 0.5 and 0.3, so each split can look at more columns. Along with this we add the count features and the duration ratio, and re-check the log-price Ridge on the new columns. 5-fold cross-validation is also carried out here like for the previous model evaluations.

Candidate	RMSE	MAE	R^2
Random forest (1000, 50% features, leaf 3, extended)	118.15	43.39	0.7160
Random forest (800, 30% features, leaf 2, extended)	119.63	44.78	0.7090
Random forest (1000, sqrt, leaf 3, extended)	120.26	49.90	0.7058
Random forest (400, sqrt, leaf 3, extended)	120.33	49.91	0.7054
Random forest (400, sqrt, leaf 3, base features)	121.37	50.97	0.7003
Log-price Ridge (deg 2, $\lambda = 10$, extended)	127.00	54.98	0.6718

Table 3: Candidates in the second tuning round.

Raising max_features to 0.5 with 1000 trees gives the real gain: the RMSE drops from 121.37 to 118.15 and the MAE from 51.0 to 43.4, so the model also tracks typical fares much better. The count features on their own only give a small improvement (121.37 to 120.33), and doubling the trees without touching anything else changes nothing measurable (120.26), which is the expected saturation. The log-price Ridge does worse once the count columns are added, so the random forest keeps carrying the comparison against the linear family.

We then test the tuned random forest on the 20% of the training rows we kept aside, and also refit it on a much smaller subset to see how much it depends on the training volume:

Model	Training rows	RMSE	MAE	R^2
Random forest (1000, 50% features, leaf 3)	80%	93.53	42.32	0.7822
Random forest (1000, 50% features, leaf 3)	20%	104.42	54.26	0.7285
Log-price Ridge (deg 2, $\lambda = 10$)	80%	110.00	53.72	0.6987
Log-price Ridge (deg 2, $\lambda = 10$)	20%	109.00	58.44	0.7042

Table 4: Scores on the unseen 20% of the training data.

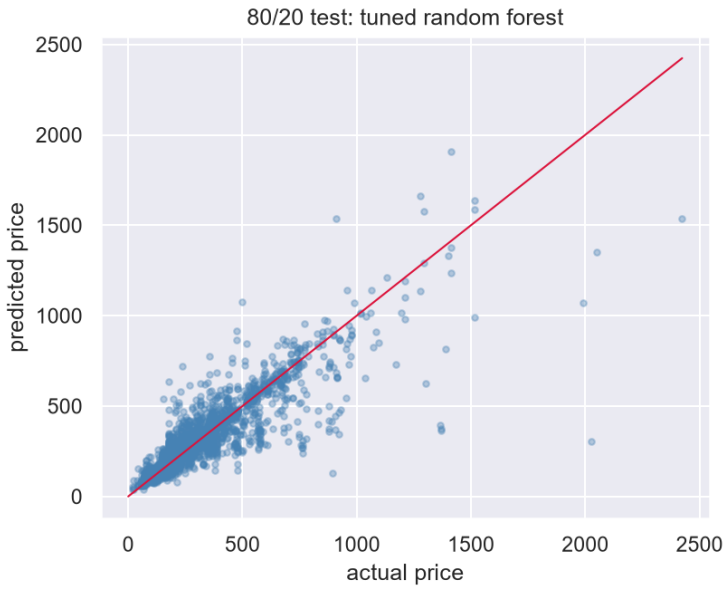


Figure 10: The tuned random forest on the unseen 20% of the training data.

When we train it on 80% of the rows the random forest reaches R^2 0.782 on the 20% it has never seen, against 0.729 for the same model trained on only 20% of the rows, so it still benefits from data volume. The Ridge moves in the other direction but stays clearly behind, and the leaderboard move agrees with the cross-validation direction: the tuned random forest raised the best public score from 0.75521 to 0.75904.

17 Conclusion

- The target is heavily skewed, so the squared error is sensitive to the rare expensive tickets and the cross-validation scores carry a wide spread.
- Cleaning removed 25 rows with a price of zero, and the engineered flight features (airline code, flight number, ordinal stops, route) all appear in the random forest's importance ranking.
- The unpenalised polynomial expansion overfits badly (degree 3 gives 380.9 RMSE), while a small L2 penalty turns degree 2 into the best raw-price linear model (136.7 RMSE). Lasso reaches a similar level and zeroes out about a third of the encoded features.
- For the tree, pre-pruning works better than cost-complexity post-pruning (128.8 against 148 RMSE).
- Training on log(price) improves the linear models further (125.0 RMSE, R^2 about 0.68); averaging the log-price Ridge with the random forest gives about 120.0 RMSE, a gap smaller than the score variation, so the simpler random forest remains the final choice.
- We also tried the random forest with more trees and max_features at 0.5 together with the new count features. That brought the RMSE down to 118.2 and the MAE to 43, and the submission scores 0.75904 on the public leaderboard.